

Operating System Lab6

禹相祐 (2312900) 查科言 (2312189) 董丰瑞 (2311973)

分工:

练习0: 董丰瑞

练习1: 查科言

练习2: 禹相祐

Challenge1: 查科言

Challenge2: FIFO 查科言 SJF 董丰瑞

报告撰写: 查科言

Operating System Lab6

- 一、练习0: 填写已有实验
- 二、练习1: 理解调度器框架的实现
 1. 调度器框架分析
 - (1) sched_class
 - (2) run_queue
 - (3) 调度器框架函数分析
 - 1) sched_init()
 - 2) wakeup_proc()
 - 3) schedule()
 2. 调度器使用流程
 - (1) 调度类的初始化流程
 - (2) 进程调度流程
 - (3) 调度算法的切换机制
- 三、练习2: 实现 Round Robin 调度算法
 1. Lab5和Lab6对比
 2. RR函数实现
 - (1) RR_init
 - (2) RR_enqueue
 - (3) RR_dequeue
 - (4) RR_pick_next
 - (5) RR_proc_tick
 3. make grade结果
 4. 实验分析
 5. 拓展思考
- 四、扩展练习 Challenge 1: 实现 Stride Scheduling 调度算法
 1. 代码切换点
 2. Stride 调度算法核心思想
 3. 数据结构与关键字段
 4. 接口实现
 - (1) stride_init
 - (2) stride_enqueue
 - (3) stride_dequeue
 - (4) stride_pick_next
 - (5) stride_proc_tick
 5. make grade 结果
 6. 为什么“运行足够久后, 时间片份额 $\propto$ priority”

- 7. 设计/实现过程简述
- 8. 补充：多级反馈队列调度算法 MLFQ 的概要设计)
- 五、扩展练习 Challenge 2：在 ucore 上实现尽可能多的各种基本调度算法 (FIFO, SJF,...)
 - 1. FIFO 调度 (First-In-First-Out)
 - 2. SJF 调度 (Shortest Job First, 实验内的近似实现)
 - 3. make qemu测试
 - 4. RR / Stride / FIFO / SJF 的适用范围与优缺点
- 六、总结：重要知识点与 OS 原理对应
 - 1. 本实验中重要的知识点
 - 2. OS 原理中重要但本实验没有直接对应的知识点

一、练习0：填写已有实验

本实验依赖实验2/3/4/5。请把你做的实验2/3/4/5的代码填入本实验中代码中有“LAB2”/“LAB3”/“LAB4”/“LAB5”的注释相应部分。并确保编译通过。注意：为了能够正确执行lab6的测试应用程序，可能需对已完成的实验2/3/4/5的代码进行进一步改进。由于我们在进程控制块中记录了一些和调度有关的信息，例如Stride、优先级、时间片等等，因此我们需要对进程控制块的初始化进行更新，将调度有关的信息初始化。同时，由于时间片轮转的调度算法依赖于时钟中断，你可能也要对时钟中断的处理进行一定的更新

我们需要补充的之前实验的代码有：

- kern/process/proc.c：
 - 函数 alloc_proc
 - 函数 do_fork
 - 函数 load_icode
- kern/trap/trap.c：
 - 函数 interrupt_handler

而Lab6这次实验需要对之前实现的部分进行修改与扩充。

这是Lab6在 proc_struct 中的新增字段，如下：

```
1 struct run_queue *rq;
2 list_entry_t run_link;
3 int time_slice;
4 skew_heap_entry_t lab6_run_pool;
5 uint32_t lab6_stride;
6 uint32_t lab6_priority;
```

我们按字段简短说明：

- struct run_queue *rq：指向当前所属的运行队列（就绪队列），不在队列时为 NULL。
- list_entry_t run_link：进程挂在运行队列链表上的节点（RR/FIFO/SJF 等用的双向链表结点）。
- int time_slice：剩余时间片，时钟中断里递减，到 0 时触发需要重新调度。
- skew_heap_entry_t lab6_run_pool：Stride 调度用的“斜堆节点”，用来把进程插入到 >lab6_run_pool 这个优先队列中。

- `uint32_t lab6_stride` : Stride 调度的累计步长, 越小越优先; 每次被选中都按 `BIG_STRIDE / lab6_priority` 增加。
- `uint32_t lab6_priority` : Stride 调度的权重, 越大表示优先级越高 (对应的 `lab6_stride` 增长越慢, 占用 CPU 比例越大)。

我们在 `proc.c` 中的 `alloc_proc` 进行了初始化:

```
1 proc->rq = NULL;
2 list_init(&(proc->run_link));
3 proc->time_slice = 0;
4 skew_heap_init(&(proc->lab6_run_pool));
5 proc->lab6_stride = 0; //stride的调度步长, 数值越小, 优先级越高
6 proc->lab6_priority = 1; //Stride调度的权重优先级,
```

我们先把“调度相关状态”清成一个安全的初始态, 后续真正入队/调度时再由调度器按算法赋值和更新。

- `proc->time_slice = 0` 初生进程不直接给时间片, 等调度器 `enqueue` 时按 `rq->max_time_slice` 统一发放。
- `skew_heap_init(&(proc->lab6_run_pool))` : 把 stride 用的斜堆节点清零, 保证后面做堆插入/删除时结构正确。
- `proc->lab6_stride = 0` : 所有新进程从同一 stride 起点开始, 后续“谁被调度一次谁就涨一截”, 由算法逐步拉开差距。
- `proc->lab6_priority = 1` : 我们默认给一个普通优先级; 以后可以通过 `lab6_set_priority` 调高或调低, 影响 stride 增长速率和 CPU 占比。

以及写好了一个 `lab6_set_priority` 函数

```
1 // FOR LAB6, set the process's priority (bigger value will get more CPU time)
2 void lab6_set_priority(uint32_t priority)
3 {
4     cprintf("set priority to %d\n", priority);
5     if (priority == 0)
6         current->lab6_priority = 1;
7     else
8         current->lab6_priority = priority;
9 }
```

在 `kern/trap/trap.c` 中, 我们在 `IRQ_S_TIMER` 分支后加了用于 LAB6 的调度钩子注释和代码。

```
1 {
2     clock_set_next_event();
3     ticks++;
4     if (ticks % TICK_NUM == 0)
5     {
6         print_ticks();
7         if (++num == 10)
8         {
9             sbi_shutdown();
10        }
11    }
12    if (current != NULL)
13    {
```

```

14         // lab6: 在时钟中断中驱动调度器进行时间片处理
15         sched_class_proc_tick(current);
16     }
17 }

```

`sched_class_proc_tick(current)` 把“发生了一次时钟中断”这个事件交给当前调度类处理

二、练习1：理解调度器框架的实现

1. 调度器框架分析

(1) sched_class

```

1 struct sched_class
2 {
3     const char *name;
4     void (*init)(struct run_queue *rq);
5     void (*enqueue)(struct run_queue *rq, struct proc_struct *proc);
6     void (*dequeue)(struct run_queue *rq, struct proc_struct *proc);
7     struct proc_struct *(*pick_next)(struct run_queue *rq);
8     void (*proc_tick)(struct run_queue *rq, struct proc_struct *proc);
9 };

```

`sched_class` 是一个“调度类接口”，用一组函数指针把具体调度算法（RR/FIFO/SJF/Stride）封装起来：

- `const char *name`：用于标识当前调度算法，方便 `sched_init` 打印、调试和性能脚本识别。
- `void (*init)(struct run_queue *rq)`：初始化运行队列，不同算法在这里搭建自己的数据结构：
 - RR/FIFO/SJF：初始化 `rq->run_list` 为空链表，`rq->lab6_run_pool = NULL`；
 - Stride：初始化 `rq->lab6_run_pool` 为空斜堆根，`run_list` 仅占位。
- `void (*enqueue)(struct run_queue *rq, struct proc_struct *proc)`：将一个已处于 `PROC_RUNNABLE` 状态的进程挂入就绪队列
 - 按算法要求插入 `rq`（链表头/尾、按 runs 排序、插入斜堆等）；
 - 初始化该进程的 `proc->rq`、`proc->time_slice`；
 - `rq->proc_num++`。
- `void (*dequeue)(struct run_queue *rq, struct proc_struct *proc)`：把一个进程从就绪队列移除：
 - 从链表/斜堆中删除相应节点；
 - 将 `proc->rq = NULL`；
 - `rq->proc_num--`。

一般在 `schedule()` 选中下一个要运行的进程之后立刻调用。
- `struct proc_struct *(*pick_next)(struct run_queue *rq)`：从就绪队列中“选一个候选者”：
 - RR/FIFO：取链表队头；
 - SJF：在链表中线性扫描 `runs` 最小的进程；

- Stride: 返回 `rq->lab6_run_pool` 堆顶对应的进程，并在内部更新它的 `lab6_stride`。
- 这个函数只负责“选”，不负责真正删出队列，由 `schedule()` 再调用 `dequeue`。
- `void (*proc_tick)(struct run_queue *rq, struct proc_struct *proc);`
每个时钟滴答 (tick) 发生时，对当前进程执行的调度逻辑：
 - `proc->time_slice--`，为 0 时设置 `proc->need_resched = 1`;

为什么用函数指针而不是直接函数？

- 调度器框架 (`schedule`、`wakeup_proc` 等) 只依赖这组统一接口，不需要知道“具体算法内部用什么数据结构”;
- 我们可以很简单的通过切换 `sched_class` 指针，就可以在运行时选择不同调度算法，而无需改调度框架代码;
- 后续我们再实现challenge里的新代码时，新算法只需实现同样签名的一组函数，扩展和对比实验非常方便。

(2) run_queue

`run_queue` 表示“当前所有处于就绪态的进程集合”。

```
1 struct run_queue
2 {
3     list_entry_t run_list;
4     unsigned int proc_num;
5     int max_time_slice;
6     skew_heap_entry_t *lab6_run_pool;
7 };
```

- lab5 中没有明确的实现 `run_queue` 和 `sched_class`，调度逻辑都直接写在 `schedule` 和 `wakeup_proc` 里，操作的是全局链表 `proc_list`
 - 唤醒进程：在 `sched.c` 里的
`wakeup_proc(struct proc_struct *proc)`
这里没有单独的“入队函数”，只是把进程状态改成 `PROC_RUNNABLE`，因为所有进程本来就挂在全局链表 `proc_list` 上。
 - 选择下一个进程：在 `sched.c` 里的 `schedule(void)`
我们直接在 `proc_list` 上用 `list_next` 线性扫描，找到下一个 `PROC_RUNNABLE` 的进程，没有通过 `run_queue/pick_next` 这种抽象。

而lab6 中的 `run_queue`，我们实现了以下四个字段：

```
1 struct run_queue
2 {
3     list_entry_t run_list;
4     unsigned int proc_num;
5     int max_time_slice;
6     skew_heap_entry_t *lab6_run_pool;
7 };
```

- `list_entry_t run_list;`
 - 就绪队列的链表头，给 RR/FIFO/SJF 等“线性扫描/队列型”算法使用。

- `unsigned int proc_num;`
 - 当前队列中的进程数量。
- `int max_time_slice;`
 - 每个进程分配的默认时间片上限，`enqueue` 时赋给 `proc->time_slice`。
- `skew_heap_entry_t *lab6_run_pool;`
 - 仅 LAB6 使用的斜堆根指针，给 stride 这类“优先队列型”算法使用。
 - 堆中每个节点对应某个进程的 `proc->lab6_run_pool` 字段。

为什么 lab6 的 run_queue 需要“链表 + 斜堆”两种结构？

- RR/FIFO/SJF 更适合用链表表达：顺序扫描或简单队头/队尾插入即可，简单直观；
- Stride 需要按 `lab6_stride` 最小优先选取进程，本质是“按键值排序的优先队列”，用斜堆更合适；
- 一个 run_queue 同时提供链表和斜堆字段，由各个 `sched_class` 决定使用哪种结构：
 - RR/FIFO/SJF：只用 `run_list`，把 `lab6_run_pool` 置为 NULL；
 - Stride：只用 `lab6_run_pool`，`run_list` 只是为了接口统一而保留。
- 这样同一个调度框架可以在不同算法之间复用，而不需要为每个算法定义一套 run_queue 类型。

(3) 调度器框架函数分析

我们主要看 lab6 中的三个函数：`sched_init`、`wakeup_proc`、`schedule`。

1) sched_init()

```

1 void sched_init(void)
2 {
3     list_init(&timer_list);
4
5     // 在这里“选一个调度算法”：把 sched_class 指向不同的 sched_class 实例
6     sched_class = &stride_sched_class;      // Challenge1 评测切换为 Stride
7     // sched_class = &default_sched_class;    // RR
8     // sched_class = &fifo_sched_class;       // Challenge2 FIFO
9     // sched_class = &sjf_sched_class;        // Challenge2 SJF
10
11     // 绑定全局 run_queue，并设置“默认时间片”
12     rq = &__rq;
13     rq->max_time_slice = MAX_TIME_SLICE;
14     // 调度类负责根据自己的需要初始化 run_queue
15     // (RR 用链表, stride 用斜堆)
16     sched_class->init(rq);
17     printf("sched class: %s\n", sched_class->name);
18 }
```

- 实现流程：
 - 初始化 `timer_list`；
 - 选择一个调度类：

```

1 | sched_class = &stride_sched_class;          // Challenge1
2 | //sched_class = &default_sched_class;        // RR
3 | //sched_class = &fifo_sched_class;           // FIFO
4 | //sched_class = &sjf_sched_class;            // SJF

```

- 绑定全局 run_queue: `rq = &__rq; rq->max_time_slice = MAX_TIME_SLICE;`
- 调用 `sched_class->init(rq)`: 由具体算法初始化 run queue 的内部结构 (链表或斜堆);
- 打印当前使用的调度类名称。
- 解耦点:
 - `sched_init` 只操作 `sched_class` 指针和 `rq`; 初始化细节全部交给 `sched_class->init` 实现;
 - 要切换算法, 只需改一行 `sched_class = &xxx_sched_class;`。

2) wakeup_proc()

```

1 | void wakeup_proc(struct proc_struct *proc)
2 | {
3 |     assert(proc->state != PROC_ZOMBIE);
4 |     bool intr_flag;
5 |     local_intr_save(intr_flag);
6 |     {
7 |         if (proc->state != PROC_RUNNABLE)
8 |         {
9 |             // 把进程从睡眠/阻塞状态唤醒为就绪态
10 |            proc->state = PROC_RUNNABLE;
11 |            proc->wait_state = 0;
12 |            if (proc != current)
13 |            {
14 |                // 如果唤醒的不是当前进程, 就把它放入就绪队列
15 |                sched_class_enqueue(proc);
16 |            }
17 |        }
18 |        else
19 |        {
20 |            warn("wakeup runnable process.\n");
21 |        }
22 |    }
23 |    local_intr_restore(intr_flag);
24 | }

```

- 实现流程:
 - 关中断, 避免并发修改;
 - 如果进程不是 `PROC_ZOMBIE` 且当前不处于 `RUNNABLE`, 则:
 - `proc->state = PROC_RUNNABLE; proc->wait_state = 0;`
 - 若被唤醒的不是 `current`, 则调用 `sched_class_enqueue(proc)`, 即 `sched_class->enqueue(rq, proc);`
 - 恢复中断。
- 解耦点:

- `wakeup_proc` 不再关心“就绪队列是什么结构”，只负责修改进程状态并调用 `enqueue`；
- 具体把进程插在链表哪儿、还是插入斜堆，由各算法的 `enqueue` 自己决定。

3) `schedule()`

```

1 void schedule(void)
2 {
3     bool intr_flag;
4     struct proc_struct *next;
5     local_intr_save(intr_flag);
6     {
7         // 清除当前进程的“需要调度”标记
8         current->need_resched = 0;
9
10        // 如果当前进程仍然是 RUNNABLE，就重新放回就绪队列等待下次运行
11        if (current->state == PROC_RUNNABLE)
12        {
13            sched_class_enqueue(current);
14        }
15        // 由调度类从 run_queue 中挑选下一个要运行的进程
16        if ((next = sched_class_pick_next()) != NULL)
17        {
18            sched_class_dequeue(next);
19        }
20        if (next == NULL)
21        {
22            // 没有就绪进程时，回退到 idle 进程
23            next = idleproc;
24        }
25        next->runs++;
26        if (next != current)
27        {
28            // 真正完成“进程切换”（切换上下文、页表等）
29            proc_run(next);
30        }
31    }
32    local_intr_restore(intr_flag);
33 }

```

- 实现流程：
 - 关中断；
 - 清除当前进程 `current->need_resched` 标记；
 - 如果当前进程状态仍为 `PROC_RUNNABLE`，则通过 `sched_class_enqueue(current)` 把它放回 `run_queue`；
 - 调用 `sched_class_pick_next()`（即 `sched_class->pick_next(rq)`）选出 `next`：
 - 若非 `NULL`，则再调用 `sched_class_dequeue(next)` 将其从 `run_queue` 移除；
 - 若为 `NULL`，则退化为 `next = idleproc`。
 - `next->runs++`；若 `next != current`，则调用 `proc_run(next)` 完成上下文切换；
 - 恢复中断。
- 解耦点：

- `schedule` 完全通过 `sched_class` 的三个钩子工作: `enqueue` / `pick_next` / `dequeue` ;
- 不关心内部是“轮转队列”还是“斜堆优先队列”, 也不关心如何比较优先级;
- 这使得更换算法不影响调度框架主逻辑。

2. 调度器使用流程

(1) 调度类的初始化流程

从内核启动到调度器就绪, 大致流程:

1. 内核早期初始化阶段完成内存、陷阱门、中断等基本设置;
2. 进程子系统 `proc_init()` 建立 `idleproc` 和 `initproc` ;
3. `sched_init()` 被调用:
 - 初始化软定时器;
 - 选择具体调度类 (实验中可选 RR/FIFO/SJF/Stride) 并赋给 `sched_class` ;
 - 将全局的 `rq` 指向静态 `__rq` , 设置 `rq->max_time_slice` ;
 - 调用 `sched_class->init(rq)` , 由相应调度算法初始化 `run_queue` (例如:
 - `default_sched_class` : 初始化链表 `run_list`、清空 `lab6_run_pool` ;
 - `stride_sched_class` : 初始化斜堆 `lab6_run_pool`、`proc_num` 等) ;
 - 打印当前调度类名称。

这样, `default_sched_class` / `fifo_sched_class` / `sjf_sched_class` / `stride_sched_class` 都是通过同一个 `sched_init` 挂接进来的, 框架只依赖其函数指针接口。

(2) 进程调度流程

可以概括为“一条从时钟中断到 `schedule` 的链路”, 关键步骤如下:

1. 时钟中断触发

- RISC-V 触发 `IRQ_S_TIMER` 中断, 硬件跳转到内核陷阱入口, 最终调用到 `interrupt_handler(tf)` 。

2. 时钟中断处理 (kern/trap/trap.c)

- 在 `case IRQ_S_TIMER`: 分支中:
 - `clock_set_next_event()`; : 设定下一次时钟中断;
 - `ticks++` 并根据 `TICK_NUM` 打印 "100 ticks"、关机等 (LAB3 逻辑) ;
 - LAB6 补充:

```
1  if (current != NULL) {
2      sched_class_proc_tick(current);
3  }
```

把本次 tick 通知调度器。

3. `proc_tick` 被调用

- `sched_class_proc_tick(current)` 内部:

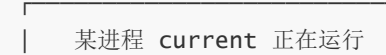
- #### 4. 从 trap 返回前检查 need_resched (kern/trap/trap.c 中 trap())

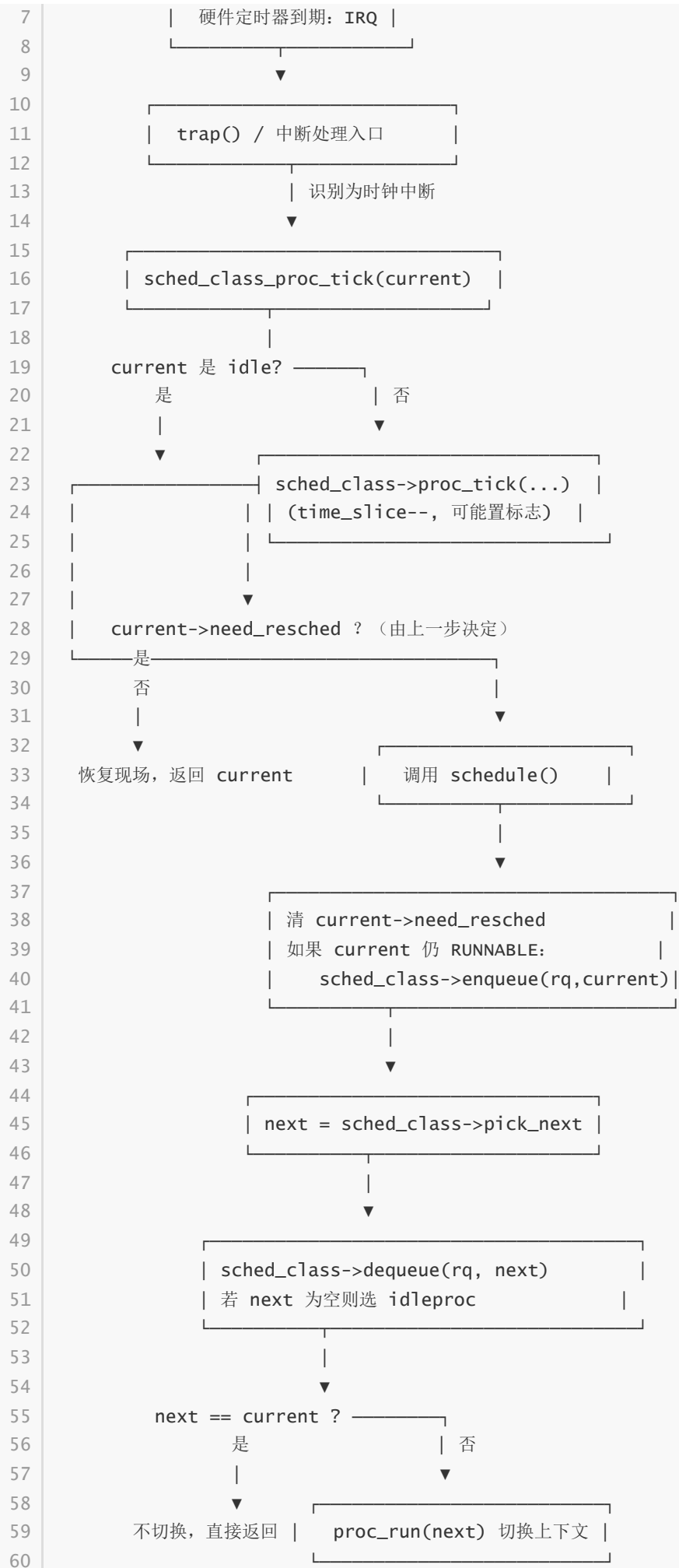
- ```
1 if (!in_kernel) {
2 if (current->flags & PF_EXITING) do_exit(...);
3 if (current->need_resched) schedule();
4 }
```

- ## 5. `schedule()` 执行调度

- ## 6. 具体调度类函数的调用顺序

- ### need\_resched 标志位的作用

- 1 | 



### (3) 调度算法的切换机制

如果要添加一个新的调度算法（例如 stride），需要做什么？

1. 首先定义一个新的 `sched_class` 实例
  - 实现对应的 `init/enqueue/dequeue/pick_next/proc_tick` 函数；
  - 设置 `.name` 为算法名字；
2. 在 `sched_init()` 中选择这个类：
  - 把 `sched_class = &sjf_sched_class`；改成 `sched_class = &stride_sched_class`；即可。
3. 如算法需要额外的 per-proc 字段，在 `struct proc_struct` 中增加字段，并在 `alloc_proc` 里初始化（LAB6 已经为 stride 加了 `lab6_run_pool/lab6_stride/lab6_priority` 等字段）。

为什么当前设计切换算法很容易？

- 调度框架只通过 `sched_class` 暴露的五个函数指针与算法交互：  
`init/enqueue/dequeue/pick_next/proc_tick`；
- `run_queue` 预留了链表和斜堆两种基础结构，基本覆盖了常见算法的需要；
- 算法内部完全封装在自己的 `.c` 文件里，不需要修改 `wakeup_proc`、`schedule` 等框架代码；
- 切换算法只改一行 `sched_class = &xxx`；其他使用逻辑（时钟中断 → `proc_tick` → `need_resched` → `schedule`）完全复用。

## 三、练习2：实现 Round Robin 调度算法

### 1. Lab5和Lab6对比

对比函数选择：我选择比较 `kern/schedule/sched.c` 中的 `schedule` 函数（lab5 和 lab6 都存在，但实现方式明显不同）。

```
1 do
2 {
3 if ((le = list_next(le)) != &proc_list)
4 {
5 next = le2proc(le, list_link);
6 if (next->state == PROC_RUNNABLE)
7 {
8 break;
9 }
10 }
11 } while (le != last);
12 if (next == NULL || next->state != PROC_RUNNABLE)
13 {
14 next = idleproc;
15 }
16 next->runs++;
17 if (next != current)
18 {
19 proc_run(next);
20 }
```

- Lab5 中的实现特点：
  - 调度逻辑直接写在 `schedule` 内部，使用的是全局进程链表，按固定策略在链表上顺序查找下一个 RUNNABLE 进程。
  - 不区分“调度框架”和“调度算法”，整个系统只能使用这一种写死的策略，改一种算法就要改 `schedule` 本身。
- Lab6 中的实现特点：
  - 新增了 `struct sched_class` 和 `struct run_queue` 抽象，`schedule` 不再直接操作链表，而是只通过 `sched_class->enqueue/dequeue/pick_next/proc_tick` 这组函数指针与调度算法交互。
  - `schedule` 的核心流程变成：“当前进程需要重入队则调用 `enqueue`，然后通过 `pick_next` 选出下一个进程，再用 `dequeue` 将其从队列结构中移除，最后调用 `proc_run` 切换”。
- 这样修改的原因：
  - 把“调度框架”（何时调度、如何切换）和“调度策略”（如何组织就绪队列、如何选择 next）解耦，便于在同一框架下切换不同调度算法（RR、FIFO、SJF、Stride 等）。
  - 后续实验只需要在各自的调度实现文件里填充 `sched_class` 的几个接口函数，而不必改动核心的 `schedule`，大大降低了实验难度和出错概率。
- 如果不做这样的改动会有什么问题：
  - 每引入一种新的调度算法都要直接修改 `schedule`，容易把已有逻辑改坏，维护成本高。
  - 无法同时支持多种调度算法并方便评测切换，也不利于从“框架层面”理解调度器的整体工作原理。

## 2. RR函数实现

### (1) RR\_init

```

1 static void
2 RR_init(struct run_queue *rq)
3 {
4 // LAB6: 2312900
5 list_init(&rq->run_list);
6 rq->proc_num = 0;
7 /* Ensure a sensible default time slice if caller didn't set it */
8 if (rq->max_time_slice <= 0)
9 rq->max_time_slice = 1;
10 rq->lab6_run_pool = NULL;
11 }

```

我们的目标是把 `run_queue` 初始化成“空的就绪队列”，让后续的入队、出队都在一个干净的状态上进行。

- 核心做法：
  - 把 `run_list` 初始化成空的双向循环链表头，这样后续所有就绪进程都以它为哨兵结点挂上去。
  - 把 `proc_num` 置 0，明确当前队列里一个进程都没有。
  - 检查 `max_time_slice`，如果外面没设置或误设成非正数，就强制设成 1，保证后面分给进程的 `time_slice` 不会是 0。

- 把 `lab6_run_pool` 设成 `NULL`，表示 RR 调度不使用堆之类的复杂结构，只用链表。

`run_queue` 成为一个简单的“空队列 + 合理默认时间片”的状态，后面所有 RR 操作都只需要围绕这几个字段工作。

## (2) RR\_enqueue

```
1 static void
2 RR_enqueue(struct run_queue *rq, struct proc_struct *proc)
3 {
4 // LAB6: 2312900
5 /* proc should not already belong to a run-queue */
6 assert(proc->rq == NULL);
7 /* insert at tail: add before the head sentinel */
8 list_add_before(&rq->run_list, &proc->run_link);
9 /* initialize process scheduling fields; protect against max_time_slice
 == 0 */
10 proc->time_slice = (rq->max_time_slice > 0) ? rq->max_time_slice : 1;
11 proc->rq = rq;
12 rq->proc_num++;
13 }
```

我们要把一个变成 `RUNNABLE` 的进程按“先来先服务”的顺序插入到就绪队列尾部，并初始化它的调度相关字段。

- 核心做法：
  - 先断言这个进程目前不在任何 `run_queue` 里，避免同一进程被插入多次导致链表结构乱掉。
  - 使用“在链表头结点前插入”的操作，相当于插到队尾；因为 `run_list` 是循环链表，`head` 前一个就是尾结点，这样就实现了 FIFO 语义。
  - 给进程分配时间片：直接设置为 `run_queue` 的 `max_time_slice`（若 `max_time_slice` 有问题再兜底成 1），保证新入队的进程从一个完整时间片开始计数。
  - 记录好双向关系：`proc->rq` 指向当前 `run_queue`，`rq->proc_num++` 统计队列长度。

这样就就绪队列按进入顺序排成一条链，每个进程都带着一整段可用的时间片等待被调度运行。

## (3) RR\_dequeue

```
1 static void
2 RR_dequeue(struct run_queue *rq, struct proc_struct *proc)
3 {
4 // LAB6: 2312900
5 /* only remove if proc currently belongs to this run-queue */
6 assert(proc->rq == rq);
7 list_del_init(&proc->run_link);
8 proc->rq = NULL;
9 if (rq->proc_num > 0)
10 rq->proc_num--;
11 }
```

当一个进程不再属于就绪队列（比如被选中运行、进入睡眠或退出）时，该函数要把他从链表中安全地把它摘掉。

- 核心做法：

- 先通过断言确认这个进程当前确实挂在这个 run\_queue 上，避免从错误的队列里删东西。
- 调用链表删除操作，把该进程对应的链表结点从 run\_list 中移除，并把它的 prev/next 指针重置到“已初始化”状态，避免悬挂指针。
- 清掉 proc->rq，把它标记为“不在任何就绪队列中”。
- 如果 proc\_num 大于 0，就自减 1，维持队列长度统计的正确性。

我们保证了无论是被调度出去还是被阻塞/退出的进程，都能干净地离开 RR 队列，不会破坏其他就绪进程的链表结构。

#### (4) RR\_pick\_next

```

1 static struct proc_struct *
2 RR_pick_next(struct run_queue *rq)
3 {
4 // LAB6: 2312900
5 if (list_empty(&rq->run_list))
6 return NULL;
7 list_entry_t *le = list_next(&rq->run_list);
8 return le2proc(le, run_link);
9 }

```

- 该函数实现在需要选出“下一个要运行的进程”时，按 Round Robin 的规则，从队头取出队首进程。
- 核心做法：
  - 先判断 run\_list 是否为空，如果空则说明当前没有普通就绪进程，返回 NULL，交给上层选择 idle 进程。
  - 如果不空，就取链表头结点的下一个结点 (head->next)，这是当前队列中“最早入队但尚未轮完一圈”的进程。
  - 再把这个链表结点转换成对应的 proc\_struct 指针返回，让 schedule 去做后续的 dequeue 和 context switch。
- 效果：RR\_pick\_next 每次都从队头取一个进程出来，配合时间片用完后重新入队，就构成了典型的“排队轮流上 CPU”的行为。

#### (5) RR\_proc\_tick

```

1 static void
2 RR_proc_tick(struct run_queue *rq, struct proc_struct *proc)
3 {
4 // LAB6: 2312900
5 if (!proc)
6 return;
7 /* consume one time slice if any left */
8 if (proc->time_slice > 0)
9 proc->time_slice--;
10 /* if time slice exhausted, request reschedule */
11 if (proc->time_slice == 0)
12 proc->need_resched = 1;
13 }

```

- 该函数实现在时钟中断到来时，对当前正在运行的进程做一次“时间片记账”，并在时间片用完时通知调度器准备抢占。

- 核心做法：
  - 先判断传进来的进程指针是否为空，防御性地避免空指针操作。
  - 如果当前进程的 `time_slice` 还大于 0，就减 1，表示已经用掉了一个时钟节拍。
  - 如果减完之后 `time_slice` 正好变成 0，就把这个进程的 `need_resched` 置 1，发出“需要调度”的信号。
- 效果：
  - 每个被 CPU 选中的进程在自己的时间片内可以连续运行多个时钟周期，一旦时间片耗尽，下一次 `trap` 收尾时就会因为 `need_resched` 被置 1 而进入 `schedule`，从就绪队列里换上下一个进程，实现真正的“时间片轮转抢占”。

### 3. make grade结果

```
100 CLICKS
● keyan@keyan-u22:/opt/riscv/code/labcode/lab6/lab6$ make grade
priority: (2.2s)
 -check result: OK
 -check output: OK
Total Score: 50/50
○ keyan@keyan-u22:/opt/riscv/code/labcode/lab6/lab6$
```

我们的实验结果如上。

我们执行 `make qemu` 之后可以看到：采用的是 `RR_scheduler` 策略

```
check_boot_pgdir() succeeded!
use SLOB allocator
kmalloc_init() succeeded!
check_vma_struct() succeeded!
check_vmm() succeeded.
sched class: RR_scheduler
++ setup timer interrupts
kernel execve: pid = 2, name = "priority".
```

### 4. 实验分析

- Round Robin 调度算法的优点：
  - 实现简单，只依赖一个 FIFO 就绪队列和时间片计数，易于在内核中维护。
  - 对所有普通进程一视同仁，每个进程都能在固定时间内获得 CPU 使用权，避免长作业完全饿死短作业。
  - 在交互式系统中，可以通过合理设置时间片大小，使得用户感觉“多个程序在并行运行”。
- Round Robin 的缺点：
  - 不考虑进程的优先级或不同作业的紧迫程度，重要任务可能和后台任务被平等对待。
  - 对 CPU 密集型任务较多的场景，频繁的上下文切换会带来额外开销，影响整体吞吐量。
  - 对 I/O 密集型进程，如果时间片设置过大，可能在一次时间片内就很快阻塞，导致时间片利用不充分。
- 时间片大小对系统性能的影响：
  - 时间片过大：



- 上下文切换次数少，切换开销小，有利于提高 CPU 利用率。
  - 但响应时间变长，交互延迟增加，用户感觉“卡顿”。
- 时间片过小：
  - 响应速度快，看起来“很公平”，进程切换也很频繁。
  - 但上下文切换开销占比显著增大，实质上浪费了大量 CPU 时间在保存/恢复现场上，整体吞吐量下降。
- 实际系统设计中通常在“响应时间”和“上下文切换开销”之间选择一个折中值。
- **为什么需要在 `RR_proc_tick` 中设置 `need_resched`：**
  - 调度器框架的设计是：`proc_tick` 只负责在“时钟中断上下文”中做时间片记账，并通过设置 `need_resched` 告诉上层“现在需要调度了”。
  - `trap()` 在中断返回前检查 `current->need_resched`，为真时才调用 `schedule()`，从而在统一的地方完成上下文切换。
  - 如果不在 `RR_proc_tick` 中设置 `need_resched`，那么时间片虽然被扣到 0，但 `trap()` 不会触发 `schedule()`，当前进程就会继续运行，失去了 Round Robin 抢占调度的意义。

## 5. 拓展思考

- **如果要实现优先级 RR 调度，代码需要如何修改？**
  - 可以在 `proc_struct` 中增加一个优先级字段（例如 `prio`），或者复用实验中已有的 `tab6_priority` 字段。
  - 在 `RR_enqueue` 中，不再简单地将进程插入链表末尾，而是按照优先级分层维护多个队列：
    - 一种方式是每个优先级维护一个单独的 `run_queue`，调度时总是先在高优先级队列中 `pick_next`。
    - 另一种方式是在单个 `run_queue` 中按“优先级高的在前”插入，同时保持同优先级内部仍采取 RR 轮转。
  - 在 `RR_pick_next` 中，优先从高优先级队列或链表前端选取进程，实现“高优先级 + 轮转”的综合策略。
- **当前实现是否支持多核调度？**
  - 目前的实验环境实际上是单核模型：
    - 内核只维护一个全局的 `run_queue` 和一个全局当前进程指针 `current`。
    - 调度决策默认只有一个 CPU 在执行，所有代码都以“单核互斥”为前提。
  - 因此，从严格意义上讲，当前实现并不支持真正的多核调度。
- **如果要支持多核调度，需要如何改进？**
  - 为每个 CPU 维护一个本地 `run_queue` 和一个本地 `current` 指针，实现 per-CPU 调度器。
  - 需要在调度器中加入锁或其他同步原语，确保多个 CPU 在迁移进程、负载均衡时不会破坏队列结构。
  - 考虑负载均衡策略：
    - 定期从负载较重的 CPU 上“偷”一部分就绪进程到负载较轻的 CPU 上（work stealing）。
    - 对绑定 CPU（CPU affinity）的进程要尽量优先放回原 CPU 的 `run_queue`。
  - 时钟中断与 `proc_tick` 也需要变为 per-CPU：
    - 每个 CPU 上的时钟中断只负责给本 CPU 的当前进程扣减时间片，并在本地触发调度，不再共享单一的全局状态。

## 四、扩展练习 Challenge 1: 实现 Stride Scheduling 调度算法

Challenge1 目标：在 lab6 的调度框架下实现 Stride Scheduling，并通过用户态测试程序（如 `priority/bench_priority`）验证“CPU 份额随优先级变化”。

### 1. 代码切换点

我们完成 Stride 调度器后，需要在 `sched_init()` 中将调度类切换为 `stride_sched_class`（与实验指导给出的切换方式一致）。系统启动时会打印：

```
1 | sched class: stride_scheduler
```

该输出用于确认：当前内核已经从默认调度类切换到我们实现的 Stride 调度类，后续的 `enqueue/dequeue/pick_next/proc_tick` 都会走 Stride 的实现。

### 2. Stride 调度算法核心思想

Stride Scheduling 的核心目标是：**让不同优先级（权重）的进程长期获得与权重成比例的 CPU 时间**，并且实现方式尽量简单、可预测。

我们用一个“累计步长” `lab6_stride` 来表示进程已经“消耗/累计”了多少 CPU 服务量：

- 每个进程有一个权重（优先级） `priority`（本实验中对应 `proc->lab6_priority`）。
- 调度器每次选择 **当前累计步长最小** 的进程运行。
- 一旦某个进程被选中运行一次，我们就把它的累计步长增加一截：

```
[
 \text{stride} \leftarrow \text{stride} + \frac{\text{BIG_STRIDE}}{\text{priority}}
]
```

直观理解是：

优先级越大（权重越高），每次被服务后增长的步长越小，因此它更容易在下一轮仍保持较小的 stride，从而更频繁地被选中；优先级越小则相反。

在本 lab 中我们采用：

- `BIG_STRIDE = 1000000` 作为缩放常量，避免浮点数运算带来的开销与不确定性；
- `rq->lab6_run_pool`（斜堆 skew heap）作为“按 `lab6_stride` 取最小值”的优先队列结构，实现高效的最小值选择。

### 3. 数据结构与关键字段

Stride 调度依赖 `proc_struct` 中的三个字段：

- `lab6_priority`：进程权重（越大意味着应获得越多 CPU 份额）
- `lab6_stride`：累计步长（越小越优先被调度）
- `lab6_run_pool`：斜堆节点（用于将该进程挂入 `rq->lab6_run_pool`）

同时我们复用 lab6 统一调度框架中的通用字段与链路：

- `proc->time_slice`：当前剩余时间片
- `proc->need_resched`：时间片耗尽时置 1，触发 `schedule()`
- `run_queue`：维护可运行进程集合（Stride 中以 skew heap 为主）

这样做的好处是：我们只需要实现 `sched_class` 要求的一组接口，就能“无缝嵌入”lab6 的调度框架，而不需要改动 `schedule()` 的整体流程。

## 4. 接口实现

Stride 的关键实现点是：用 `lab6_stride` 做最小键值，并且在每次选择某进程运行后对其 `lab6_stride` 做一次“确定性递增”。

下面给出我们实现中的核心常量、比较器，以及五个接口函数。

```
1 /* BIG_STRIDE: a large constant used to scale strides. */
2 #define BIG_STRIDE 1000000
3
4 /* The compare function for two skew_heap_node_t's and the corresponding
 procs */
5 static int
6 proc_stride_comp_f(void *a, void *b)
7 {
8 struct proc_struct *p = le2proc(a, lab6_run_pool);
9 struct proc_struct *q = le2proc(b, lab6_run_pool);
10 int32_t c = p->lab6_stride - q->lab6_stride;
11 if (c > 0)
12 return 1;
13 else if (c == 0)
14 return 0;
15 else
16 return -1;
17 }
```

比较器含义很直接：`lab6_stride` 小的进程在堆中更“靠前”，最终堆顶就是 stride 最小者。

### (1) stride\_init

目标：初始化运行队列，使其处于“空队列”状态。

实现要点：

- `rq->run_list` 仍初始化为空（保持与框架一致，虽然 Stride 不主要依赖链表）
- `rq->lab6_run_pool = NULL`：斜堆为空
- `rq->proc_num = 0`：可运行进程数清零

```

1 static void
2 stride_init(struct run_queue *rq)
3 {
4 list_init(&rq->run_list);
5 rq->lab6_run_pool = NULL;
6 rq->proc_num = 0;
7 }

```

## (2) stride\_enqueue

目标：将一个 `PROC_RUNNABLE` 进程加入就绪队列（优先队列）。

实现要点：

- 我们首先断言 `proc->rq == NULL`，避免同一进程被重复入队（重复入队会破坏堆结构与计数）。
- 与 RR 一致，我们在入队时为其初始化时间片 `proc->time_slice`，保证调度与抢占机制能够正常工作。
- 关键操作：把 `&proc->lab6_run_pool` 插入 `rq->lab6_run_pool`（斜堆插入），使该进程参与“按 stride 取最小”的竞争。

```

1 static void
2 stride_enqueue(struct run_queue *rq, struct proc_struct *proc)
3 {
4 assert(proc->rq == NULL);
5 rq->lab6_run_pool = skew_heap_insert(rq->lab6_run_pool, &proc-
6 >lab6_run_pool, proc_stride_comp_f);
7 proc->time_slice = (rq->max_time_slice > 0) ? rq->max_time_slice : 1;
8 proc->rq = rq;
9 rq->proc_num++;
10 }

```

## (3) stride\_dequeue

目标：把进程从就绪队列移除（通常由 `schedule()` 在选中进程后调用）。

实现要点：

- 断言 `proc->rq == rq`，保证“从哪个队列入，就从哪个队列出”。
- 关键操作：从斜堆中删除对应节点。
- 更新 `proc->rq = NULL` 并维护 `rq->proc_num`，保证队列状态一致。

```

1 static void
2 stride_dequeue(struct run_queue *rq, struct proc_struct *proc)
3 {
4 assert(proc->rq == rq);
5 rq->lab6_run_pool = skew_heap_remove(rq->lab6_run_pool, &proc-
6 >lab6_run_pool, proc_stride_comp_f);
7 proc->rq = NULL;
8 if (rq->proc_num > 0)
9 rq->proc_num--;
10 }

```

## (4) stride\_pick\_next

目标：选择下一个要运行的进程（stride 最小者）。

实现要点：

- 如果 `rq->lab6_run_pool == NULL`，说明没有可运行进程，返回 `NULL`。
- 否则堆顶就是 stride 最小的进程：`le2proc(rq->lab6_run_pool, lab6_run_pool)`。
- **注意：**`pick_next` 只负责“挑选”，不负责“出队”。真正的删除动作由 `schedule()` 统一调用 `dequeue()` 完成，这样接口职责更清晰，也符合 lab6 框架的调用方式。
- 我们在选中后立即更新该进程的 `lab6_stride`，让它“为刚刚获得的服务付出 stride 增量”，防止其一直霸占最小值。

```
1 static struct proc_struct *
2 stride_pick_next(struct run_queue *rq)
3 {
4 if (rq->lab6_run_pool == NULL)
5 return NULL;
6 struct proc_struct *p = le2proc(rq->lab6_run_pool, lab6_run_pool);
7 uint32_t pri = p->lab6_priority;
8 if (pri == 0)
9 pri = 1;
10 p->lab6_stride += (uint32_t)(BIG_STRIDE / pri);
11 return p;
12 }
```

这样做的效果是：进程每“被服务一次”，就会累加一段 stride。短期内它的 stride 会变大，从而把“下次机会”让给其他 stride 更小的进程；而优先级更高的进程由于每次加得更少，会在长期中更频繁地成为最小者，最终体现为更高的 CPU 份额。

## (5) stride\_proc\_tick

目标：每个时钟 tick 更新当前运行进程的时间片，并在耗尽时触发调度。

实现要点与 RR 一致：

- `time_slice > 0` 时递减
- 若递减后变为 0，则设置 `need_resched = 1`，让内核在合适时机进入 `schedule()`

```
1 static void
2 stride_proc_tick(struct run_queue *rq, struct proc_struct *proc)
3 {
4 if (!proc)
5 return;
6 if (proc->time_slice > 0)
7 proc->time_slice--;
8 if (proc->time_slice == 0)
9 proc->need_resched = 1;
10 }
```

## 5. make grade 结果

我们执行 `make qemu` 得到如下结果，可以看到采用的是 `stride_scheduler` 调度算法：

在本次提交中我们切换为 `stride` 后运行 `make grade`，其中 `priority`（以及相关的优先级份额验证逻辑）能够通过，说明：**不同 `lab6_priority` 的进程确实会表现出不同的 CPU 时间片占比**，与 `Stride` 的设计目标一致。

---

## 6. 为什么“运行足够久后，时间片份额 $\propto$ priority”

我们用一个不追求形式化、但能够自洽且能说服自己的说明。

设进程  $i$  的权重为  $p_i$ ，在系统运行过程中，它被选中运行（成为 `next` 并获得一个时间片）的次数为  $n_i$ 。在 `Stride` 中每次被选中后：

$$S_i \leftarrow S_i + \frac{B}{p_i}$$

其中  $S_i$  是累计 `stride`， $B = \text{BIG\_STRIDE}$ 。

调度器每次都选择当前 ( $S$ ) 最小的进程运行。长期运行后会出现一个“自动拉平”的现象：

- 如果某个进程  $i$  的  $S_i$  长期明显小于其他进程，那么它会被更频繁地选中；
- 但被更频繁选中意味着它会反复累加  $\frac{B}{p_i}$ ，从而把  $S_i$  推高；
- 反过来，如果某个进程  $j$  的  $S_j$  较大，它就会一段时间内较少被选中，使得其它进程有机会追上来。

因此在稳定阶段，各可运行进程的  $S_i$  会处在相近的量级（不会长期差得离谱）。于是我们可以近似认为不同进程累计 `stride` 大致相当：

$$n_i \cdot \frac{B}{p_i} \approx n_j \cdot \frac{B}{p_j}$$

消去  $B$  得：

$$\frac{n_i}{n_j} \approx \frac{p_i}{p_j}$$

也就是说：**当运行足够多的时间片之后，各进程获得的时间片次数之比，近似等于它们优先级（权重）之比**。这正是我们在 `priority` 测试中希望观察到的“CPU 份额随优先级变化”的效果。

---

## 7. 设计/实现过程简述

- 我们复用 `lab6` 的统一调度框架：`sched_class` + `run_queue` + `need_resched` 的整体链路不改，只替换调度类实现。
- 我们用斜堆 (skew heap) 实现优先队列：以 `lab6_stride` 为键，保证 `pick_next` 能快速拿到 `stride` 最小的进程。
- 我们把“份额控制”的关键逻辑集中在 `pick_next` 的 `stride` 更新：通过 `BIG_STRIDE / priority` 控制增长速率，从而让不同权重在长期中体现出不同的调度频率。
- 我们在实现中保持接口职责清晰：`pick_next` 只选择不出队，实际删除由 `schedule()` 统一调用 `dequeue()` 完成，避免状态更新分散导致错误。

- 我们通过 `priority` 验证：当进程优先级不同且持续运行足够久时，进程获得的 CPU 时间片份额会随优先级变化，符合 Stride 的预期。

## 8.补充：多级反馈队列调度算法 MLFQ 的概要设计)

在同样的 lab6 框架下，如果我们要实现“多级反馈队列调度算法”，

整体思路是：把可运行进程按“交互性/CPU 占用倾向”分层，优先运行更“像交互进程”的队列，并通过用完时间片就下调、等待一段时间再上调（提升/老化）来避免饥饿。

- 我们会在 `run_queue` 中维护多个队列，例如 `N` 个 level：`q[0]` 最高优先级，`q[N-1]` 最低优先级；每个队列用链表即可（因为 MLFQ 的选择规则是“先看高优先级队列是否非空”，不需要全局最小键结构）。
- 我们会在 `proc_struct` 中新增/复用字段，记录：
  - `mlfq_level`：当前所在队列层级
  - `wait_ticks` 或 `last_run`：用于提升/老化（aging），避免低层进程长期饿死
  - 时间片大小可按层级变化：`time_slice = base << level` 或者相反（实验可选策略，只要逻辑一致）
- 接口层面，我们仍实现同一套 `sched_class`：
  - `enqueue`：进程入队到其当前 level 的队列尾部
  - `dequeue`：从对应 level 队列移除
  - `pick_next`：从最高 level 开始找第一个非空队列，取队首作为 next
  - `proc_tick`：时间片递减；若用完则 `need_resched=1`，并在下次 `enqueue` 时把进程降到更低 level（或直接在 tick 中调整，再触发调度）
  - 周期性提升：每隔一段时间扫描低层队列，把等待过久的进程上调（aging），保证公平性
- 这样设计的直观效果是：
  - 经常主动让出 CPU / 不怎么用满时间片的进程会停留在高优先级队列，响应更快；
  - 长时间占用 CPU 的进程会逐步下沉到低优先级队列，减少对交互型任务的影响；
  - 通过 aging 可以避免低优先级任务永远得不到运行机会。

以上是我们在 lab6 统一框架下对 MLFQ 的概要设计思路：接口仍遵循 `sched_class`，主要差异在于 `run_queue` 的组织方式（多队列）与“用完就降级/等待就升级”的策略实现。

## 五、扩展练习 Challenge 2：在 ucore 上实现尽可能多的各种基本调度算法（FIFO, SJF,...）

Challenge2 目标：在同一调度框架下新增 FIFO 与 SJF 两种调度算法，并设计测试用例，能够定量分析不同调度算法在关键指标上的差异，进一步说明各算法的适用范围与优缺点。

### 1. FIFO 调度（First-In-First-Out）

FIFO（先来先服务）调度的核心规则是：**就绪队列按到达顺序排队，每次总是选择最早进入队列的进程运行。**在 lab6 的调度框架下，FIFO 可以非常自然地用 `run_list`（双向链表）实现：把“先后顺序”直接交给链表的插入顺序维护即可。

在接口层面，FIFO 需要实现 `init/enqueue/dequeue/pick_next/proc_tick` 五个函数，并满足框架约束：`pick_next` 只负责“选谁”，实际移除由 `schedule()` 在切换前调用 `dequeue()` 完成，这样可以保证不同调度类的行为一致性。

FIFO 的关键实现代码如下：

```
1 static void
2 FIFO_init(struct run_queue *rq)
3 {
4 // 初始化就绪队列（双向链表头结点）
5 list_init(&rq->run_list);
6
7 // 就绪队列中的进程数量清零
8 rq->proc_num = 0;
9
10 // 时间片至少为 1，避免出现 time_slice 为 0 导致无法运行的情况
11 if (rq->max_time_slice <= 0)
12 rq->max_time_slice = 1;
13
14 // FIFO 不使用 skew heap，置空只是保持 run_queue 字段状态明确
15 rq->lab6_run_pool = NULL;
16 }
17
18 static void
19 FIFO_enqueue(struct run_queue *rq, struct proc_struct *proc)
20 {
21 // 进程必须尚未在任何 run_queue 中
22 assert(proc->rq == NULL);
23
24 // FIFO 的“先来先服务”靠队列顺序体现：
25 // 采用尾插，使更早入队的进程更靠近队头
26 list_add_before(&rq->run_list, &proc->run_link);
27
28 // 入队时为进程补齐/重置时间片，沿用统一框架的抢占机制
29 proc->time_slice = (rq->max_time_slice > 0) ? rq->max_time_slice : 1;
30
31 // 绑定所属队列，便于 dequeue 时一致性检查
32 proc->rq = rq;
33 rq->proc_num++;
34 }
35
36 static void
37 FIFO_dequeue(struct run_queue *rq, struct proc_struct *proc)
38 {
39 // 被移除的进程必须属于该队列
40 assert(proc->rq == rq);
41
42 // 从就绪队列中删除该节点，并把节点指针重置为独立状态
43 list_del_init(&proc->run_link);
44
45 proc->rq = NULL;
46 if (rq->proc_num > 0)
47 rq->proc_num--;
48 }
49
```



```

50 static struct proc_struct *
51 FIFO_pick_next(struct run_queue *rq)
52 {
53 // 无可运行进程则返回 NULL
54 if (list_empty(&rq->run_list))
55 return NULL;
56
57 // 取队头：队头是最早入队的进程
58 list_entry_t *le = list_next(&rq->run_list);
59 return le2proc(le, run_link);
60 }
61
62 static void
63 FIFO_proc_tick(struct run_queue *rq, struct proc_struct *proc)
64 {
65 // 与 RR 相同：每个时钟 tick 消耗当前进程的 time_slice
66 if (!proc)
67 return;
68
69 if (proc->time_slice > 0)
70 proc->time_slice--;
71
72 // 时间片耗尽则触发重新调度
73 if (proc->time_slice == 0)
74 proc->need_resched = 1;
75 }
76
77 struct sched_class fifo_sched_class = {
78 .name = "FIFO_scheduler",
79 .init = FIFO_init,
80 .enqueue = FIFO_enqueue,
81 .dequeue = FIFO_dequeue,
82 .pick_next = FIFO_pick_next,
83 .proc_tick = FIFO_proc_tick,
84 };

```

#### 代码解释：

- `FIFO_enqueue` 尾插：队列头永远是最早到达的进程，因此 `FIFO_pick_next` 只需返回队头即可，不需要扫描或排序。
- `FIFO_pick_next` 不做 `dequeue`：这是为了符合 lab6 框架的职责划分，避免“选中但没切换”时队列状态被提前破坏。
- `FIFO_proc_tick` 仍然使用时间片抢占：这使得 FIFO 在 ucore 中更像“按到达顺序轮流拿时间片”，而不是严格意义上“非抢占 FIFO”。这种实现更容易与框架兼容，也能避免单个进程永久占用 CPU。

适用场景：实现简单、队列操作开销低；但对交互和短任务不友好，长任务在前会显著拉高后续任务的等待时间（convoy effect）。

---

## 2. SJF 调度 (Shortest Job First, 实验内的近似实现)

SJF (最短作业优先) 的核心规则是：**每次选择“预计运行时间最短”的进程先运行**。严格 SJF 需要事先知道作业长度，但操作系统通常无法准确获得。因此这里采用一个可实现的近似：用 `proc->runs` 作为“历史运行量”的粗略估计，把 `runs` 小的进程当作更“短/更轻”的任务优先调度。

实现上仍复用 `rq->run_list` 存放就绪进程，但与 FIFO 不同的是：`pick_next` 需要线性扫描整个队列，找到 `runs` 最小的进程。

SJF 的关键实现代码如下：

```
1 static void
2 SJF_init(struct run_queue *rq)
3 {
4 // 初始化就绪队列
5 list_init(&rq->run_list);
6
7 // 可运行进程数清零
8 rq->proc_num = 0;
9
10 // 时间片至少为 1
11 if (rq->max_time_slice <= 0)
12 rq->max_time_slice = 1;
13
14 // SJF 不使用 skew heap, 置空保持状态明确
15 rq->lab6_run_pool = NULL;
16 }
17
18 static void
19 SJF_enqueue(struct run_queue *rq, struct proc_struct *proc)
20 {
21 assert(proc->rq == NULL);
22
23 // 入队仍采用尾插：不在 enqueue 阶段维护排序
24 // “最短作业”的选择推迟到 pick_next 中做
25 list_add_before(&rq->run_list, &proc->run_link);
26
27 proc->time_slice = (rq->max_time_slice > 0) ? rq->max_time_slice : 1;
28 proc->rq = rq;
29 rq->proc_num++;
30 }
31
32 static void
33 SJF_dequeue(struct run_queue *rq, struct proc_struct *proc)
34 {
35 assert(proc->rq == rq);
36 list_del_init(&proc->run_link);
37 proc->rq = NULL;
38 if (rq->proc_num > 0)
39 rq->proc_num--;
40 }
41
42 static struct proc_struct *
43 SJF_pick_next(struct run_queue *rq)
44 {
```

```

45 if (list_empty(&rq->run_list))
46 return NULL;
47
48 // runs 用作“作业长度”的近似指标：
49 // runs 越小，表示累计运行越少，优先被选择
50 list_entry_t *le = list_next(&rq->run_list);
51 struct proc_struct *best = NULL;
52
53 for (; le != &rq->run_list; le = list_next(le))
54 {
55 struct proc_struct *p = le2proc(le, run_link);
56 if (best == NULL || p->runs < best->runs)
57 {
58 best = p;
59 }
60 }
61 return best;
62 }
63
64 static void
65 SJF_proc_tick(struct run_queue *rq, struct proc_struct *proc)
66 {
67 // 与 RR 相同: tick 消耗 time_slice, 用完触发调度
68 if (!proc)
69 return;
70
71 if (proc->time_slice > 0)
72 proc->time_slice--;
73
74 if (proc->time_slice == 0)
75 proc->need_resched = 1;
76 }
77
78 struct sched_class sjf_sched_class = {
79 .name = "SJF_scheduler",
80 .init = SJF_init,
81 .enqueue = SJF_enqueue,
82 .dequeue = SJF_dequeue,
83 .pick_next = SJF_pick_next,
84 .proc_tick = SJF_proc_tick,
85 };

```

### 代码解释：

- `SJF_enqueue` 不排序而是直接尾插，优点是入队成本低、实现简单；缺点是 `pick_next` 每次都要  $O(n)$  扫描。
- `SJF_pick_next` 选择 `runs` 最小者：这更接近“Shortest Remaining/Shortest Job”的味道，但它本质上是“基于历史运行量的启发式”，并不等价于真实的作业长度预测。
- 该 SJF 近似策略的直接结果是：累计运行较少的进程会更容易被再次选中，从而改善短任务体验；但对 `runs` 已经较大的进程会越来越不利，可能出现饥饿，需要配合 aging 才能从根本上缓解。

适用场景：短任务密集、希望降低平均等待/周转时间；但公平性较弱，长任务可能等待过久。

### 3. make qemu测试

我们依旧用 make qemu 测试

```
check_vma_struct() succeeded!
check_vmm() succeeded.
sched_class: FIFO_scheduler
++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
main: fork ok,now need to wait pids.
set priority to 5
set priority to 4
set priority to 3
set priority to 2
set priority to 1
```

```
check_vmm() succeeded.
sched_class: SJF_scheduler
++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
main: fork ok,now need to wait pids.
set priority to 5
set priority to 4
set priority to 3
set priority to 2
set priority to 1
all user-mode processes have quit.
init check memory pass.
```

后续分析我们认为可以采用以下指标：

- 平均周转时间 (turnaround)
- 平均等待时间 (waiting)
- 平均响应时间 (response)
- 吞吐量 (单位时间完成数量)
- 公平性 (例如各进程 tick 数的离散程度，或与期望份额的偏差)

为了让对比更“定量”，可以对每种算法在同一组 workload 下重复运行多次，取平均值并观察方差（是否稳定、是否有抖动）。

---

### 4. RR / Stride / FIFO / SJF 的适用范围与优缺点

- **RR (Round Robin)**
  - 优点：响应时间好、基本公平、实现简单
  - 缺点：时间片过小时上下文切换开销大；无法天然支持按权重分配份额
  - 适用：交互型场景；任务长度差异较大但希望避免长期等待
- **Stride (按权重的份额调度)**
  - 优点：CPU 份额可控且接近与 priority 成比例；确定性强
  - 缺点：需要优先队列/堆结构维护 stride 最小值；参数设置不当会伤害低权重任务体验

- 适用：需要明确资源分配策略的场景（不同服务等级、不同重要性任务）
  - **FIFO（先来先服务）**
    - 优点：开销低、实现最简单、顺序直观
    - 缺点：短任务/交互体验差；长任务在前会导致整体等待时间变大（convoy effect）
    - 适用：任务长度相近、顺序本身有意义；或作为对照基线算法
  - **SJF（最短作业优先，本实验为 runs 近似）**
    - 优点：平均等待/周转时间通常更优，短任务完成更快
    - 缺点：严格 SJF 难以获得作业长度；近似实现可能不稳定；存在饥饿风险
    - 适用：批处理、短任务占多数、且能较好估计“短/长”的场景；或作为优化等待时间的对照策略
- 

## 六、总结：重要知识点与 OS 原理对应

### 1. 本实验中重要的知识点

#### 1. 调度框架抽象（`sched_class` 接口）

- 对应原理：**机制与策略分离（mechanism vs. policy）**
- 理解：框架（`schedule/wakeup_proc` 等）提供“何时切换、如何切换”的机制；RR/FIFO/SJF/Stride 是“选谁上 CPU”的策略。二者分离能让替换策略不破坏机制，降低修改风险。

#### 2. 就绪队列 `run_queue` 与进程状态流转（`RUNNABLE` 等）

- 对应原理：**进程状态机、就绪队列模型**
- 理解：实验把“可运行进程集合”显式化为 `run_queue`，把“状态变化”和“入队/出队”绑定起来。原理上这是把抽象状态机落到具体数据结构上。

#### 3. 时间片 `time_slice` + 抢占标志 `need_resched`

- 对应原理：**时钟中断驱动的抢占式调度、延迟调度（deferred scheduling）**
- 理解：在中断上下文里只做轻量记账（`time_slice--` 并置位），真正的上下文切换推迟到安全路径执行。差异在于：原理讲“抢占”通常是概念性的；实验体现为“中断里不直接切换”。

#### 4. RR（Round Robin）实现与公平性

- 对应原理：**分时系统、响应时间与上下文切换开销权衡**
- 理解：RR 用时间片实现“近似公平”，但公平不是免费的：时间片越小响应越快、切换越频繁；时间片越大吞吐可能更好、交互更差。

#### 5. FIFO 与“非抢占/弱抢占”特性

- 对应原理：**先来先服务的队列模型、饥饿与响应性问题**
- 理解：FIFO 对短任务/交互不友好，容易出现“队首长任务拖慢所有人”。实验里虽然仍用 `proc_tick` 触发调度，但策略层依旧体现了 FIFO 的核心缺陷：不区分作业特征。

#### 6. SJF 的近似实现（用 `runs` 作为“长度”估计）

- 对应原理：**SJF/预测 CPU burst、最小化平均等待时间**

- 理解：原理中的 SJF 依赖“已知或可预测的运行时间”，现实里做不到精确。实验用 `runs` 这种历史统计做近似，本质是用“过去的 CPU 使用量”替代“未来的 CPU burst”，所以只能得到倾向性效果，且仍可能饥饿。

## 7. Stride（比例份额调度）与优先级/权重

- 对应原理：**proportional-share scheduling（按份额分配 CPU）**
- 理解：Stride 用整数步长把“CPU 份额”变成可比较的标量（`stride`），长期看份额  $\propto$  权重。与“静态优先级”不同：这里优先级不是“谁永远更先跑”，而是“长期拿到多少比例”。

## 8. 优先队列（斜堆）实现 Stride 的 pick\_next

- 对应原理：**内核数据结构选择与复杂度权衡（链表 vs. 优先队列）**
- 理解：RR/FIFO/SJF 用链表就够（插入/扫描简单），Stride 需要频繁取最小 `stride`，用优先队列更合适。差异点在于：原理层通常不限定具体数据结构；实验要求你“用某种结构把策略落地”。

## 9. 用户态影响调度（设置优先级的系统调用）

- 对应原理：**系统调用接口、资源管理与权限控制**
- 理解：实验展示了“用户可以通过 `syscall` 影响内核策略参数”。但在真实系统里，调高优先级涉及权限与防滥用（否则可能造成拒绝服务），实验中一般简化为功能验证。

# 2. OS 原理中重要但本实验没有直接对应的知识点

1. **多核调度与负载均衡**：per-CPU run queue、任务迁移、work stealing、NUMA/缓存亲和性等。
2. **实时调度与时限约束**：RM/EDF、deadline、抖动（jitter）控制、可调度性分析。
3. **优先级反转与继承机制**：锁竞争下的 priority inversion、priority inheritance/ceiling。
4. **更贴近现代 OS 的通用调度器设计**：如 CFS（红黑树按虚拟运行时间）、组调度/配额（cgroups 类思想）。
5. **I/O 与调度的耦合**：阻塞/唤醒对交互性的影响、I/O completion、磁盘/网络队列与 CPU 调度的协同。
6. **能耗与温控相关调度**：DVFS、能量感知调度（energy-aware scheduling）。
7. **安全与隔离视角的资源分配**：如何限制用户态调度影响、避免恶意抢占、配额与审计。